

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А.И. ГЕРЦЕНА»



Направление подготовки/специальность
Направление подготовки 09.03.01 Информатика и вычислительная техника

Направленность (профиль) «Технологии разработки программного
обеспечения»

форма обучения – очная

Выпускная квалификационная работа

Создание инструментария для no-code разработки Telegram-ботов на Python

Обучающегося 4 курса
очной формы обучения
Киняева Ильи Константиновича

Руководитель выпускной квалификационной
работы:
доцент, кандидат педагогических наук
Атаян Ануш Михайловна

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. Теоретические основы разработки инструментария для no-code создания Telegram-ботов.....	6
1.1 Особенности разработки Telegram-ботов.....	6
1.2 Обзор существующих решений для no-code создания Telegram-ботов	8
1.3 Обзор инструментальных средств и технологий реализации no-code инструментария для Telegram-ботов	14
1.3.1 Анализ языковых средств разработки	14
1.3.2 Анализ фреймворков, библиотек и среды разработки.....	16
Выводы к главе 1	22
ГЛАВА 2. Проектирование и разработка программного продукта.....	23
2.1 Описание структуры и принципа работы программы	23
2.2 Функциональная модель и работа пользователя с системой.....	29
2.3 Проверка работы инструментария на типовых сценариях	34
Выводы к главе 2	37
ЗАКЛЮЧЕНИЕ	38
ЛИТЕРАТУРА.....	40

ВВЕДЕНИЕ

Сервисы, предоставляемые Telegram-ботами, пользуются спросом: с их помощью организации и отдельные пользователи автоматизируют информирование, консультирование, приём обращений, проведение опросов и навигацию по услугам. Telegram-боты применяются в различных сферах: в образовании, малом бизнесе, технической поддержке, информационном сопровождении пользователей и организации простых интерактивных сценариев.

Разработка Telegram-ботов обычно требует знания языка программирования, понимания принципов работы Telegram Bot API, настройки обработки сообщений, команд, кнопок, состояний и хранения данных. Для пользователей, не обладающих навыками программирования, самостоятельное создание даже простого сценарного бота может быть затруднительным. В связи с этим актуальной является задача разработки средств, позволяющих создавать Telegram-ботов без программирования и отвечающих потребностям пользователей, которым необходимы типовые сценарные боты.

Особую значимость данная задача приобретает при создании ботов, логика которых может быть представлена в виде последовательности состояний, сообщений, кнопок и переходов. К таким ботам относятся FAQ-боты, простые заявочные боты, боты для проведения тестирования, навигационные боты и другие решения, не требующие сложной бизнес-логики, но нуждающиеся в удобном механизме настройки сценария.

В рамках данной работы был разработан инструментарий для no-code создания Telegram-ботов на языке Python. Разрабатываемая система позволяет описывать логику Telegram-бота с помощью готовых элементов: состояний, узлов, триггеров, действий, сообщений и клавиатур. Такой подход позволяет пользователю формировать сценарий работы бота без ручного изменения программного кода.

Целью дипломной работы стала разработка инструментария для no-code создания Telegram-ботов.

Объектом исследования является процесс разработки Telegram-ботов.

Предметом исследования является программное обеспечение для описания логики Telegram-бота на основе состояний, узлов, триггеров, действий, сообщений и клавиатур.

Теоретическая значимость работы заключается в систематизации сведений об особенностях разработки Telegram-ботов, no-code подходе к созданию программных решений, а также об инструментальных средствах, применяемых для реализации Telegram-ботов на языке Python.

Результаты анализа существующих решений и инструментальных средств могут быть использованы при изучении подходов к разработке сценарных Telegram-ботов и no-code инструментов. Рассмотрение состояний, триггеров, действий, сообщений и клавиатур позволяет показать, каким образом логика бота может быть представлена не в виде исходного кода, а в виде набора настраиваемых элементов.

Практическая значимость работы заключается в возможности создания типовых Telegram-ботов без написания программного кода пользователем. Разработанный инструментарий может применяться для создания FAQ-ботов, простых заявочных ботов и ботов для проведения тестирования.

Для достижения поставленной цели были решены следующие задачи:

1. Проанализировать особенности разработки Telegram-ботов и существующие решения в области создания ботов без программирования
2. Проанализировать и на основе анализа отобрать адекватные задаче инструментальные средства
3. Спроектировать модель сценарного Telegram-бота в соответствии с выдвинутыми требованиями

4. В соответствии с моделью разработать инструментарий и провести проверку его работы на типовых сценариях создания Telegram-ботов.

В рамках решения последней задачи была выполнена проверка работы разработанного инструментария на сценариях создания FAQ-бота, простого заявочного бота и бота для проведения тестирования.

В качестве результата дипломной работы представлен программный продукт, позволяющий создавать и настраивать Telegram-ботов через управляющего Telegram-бота. В системе реализована возможность создания бота, настройки состояний и узлов, добавления сообщений и клавиатур, создания триггеров и действий, а также проверки работы созданного сценария.

Структура дипломной работы определяется поставленной целью и задачами. Работа состоит из введения, двух глав, заключения и списка использованных источников.

ГЛАВА 1. Теоретические основы разработки инструментария для no-code создания Telegram-ботов

1.1 Особенности разработки Telegram-ботов

Telegram-бот представляет собой программный компонент, который взаимодействует с пользователями через мессенджер Telegram. В отличие от обычного пользователя, бот управляется программой, которая принимает входящие события, обрабатывает их и отправляет ответные сообщения. Для взаимодействия с платформой Telegram используется Bot API — HTTP-интерфейс, предназначенный для разработки ботов и обмена данными между серверной частью приложения и Telegram [13].

Создание Telegram-бота начинается с регистрации через BotFather. BotFather является служебным ботом Telegram, с помощью которого создаются новые боты, задаются их параметры и выдаётся токен доступа. Токен используется серверной частью приложения для обращения к Telegram Bot API и управления созданным ботом [14].

После регистрации бота разработчику необходимо реализовать программную часть, которая будет получать события от Telegram и выполнять соответствующие действия. К таким событиям относятся отправка пользователем сообщения, ввод команды, нажатие кнопки и другие действия. В Telegram Bot API такие события передаются в виде обновлений, которые бот должен получать и обрабатывать [13].

Существует два основных способа получения обновлений от Telegram: long polling и webhook. При использовании long polling приложение периодически обращается к Telegram Bot API и проверяет наличие новых событий. При использовании webhook Telegram самостоятельно отправляет обновления на заранее указанный адрес серверного приложения. Выбор способа зависит от условий размещения приложения и требований к его работе [13].

Важной особенностью Telegram-ботов является событийная модель работы. Бот не выполняет один общий линейный сценарий для всех пользователей, а реагирует на отдельные события, поступающие от разных пользователей. Поэтому при разработке необходимо учитывать текущее состояние пользователя, выбранный этап сценария и данные, которые были получены ранее.

Для простых Telegram-ботов может быть достаточно обработки отдельных команд и сообщений. Например, команда `/start` обычно используется для начала работы с ботом, а команда `/help` — для вывода справочной информации. Однако для более сложных сценариев требуется хранить контекст взаимодействия с пользователем. Это особенно важно для заявочных ботов, тестовых ботов, FAQ-ботов и других сценариев, где пользователь проходит несколько этапов.

Telegram поддерживает различные элементы взаимодействия с пользователем: текстовые сообщения, команды, кнопки, встроенные клавиатуры и меню. Использование кнопок и клавиатур позволяет уменьшить количество ручного ввода и сделать сценарий более управляемым. Пользователь выбирает один из предложенных вариантов, а бот выполняет действие, связанное с этим выбором [15].

При разработке Telegram-ботов также необходимо учитывать хранение данных. В простых ботах данные могут ограничиваться текстами сообщений и настройками команд. В более сложных системах требуется хранить информацию о пользователях, состояниях, заявках, результатах тестирования, выбранных вариантах ответа и других элементах сценария. Поэтому при проектировании бота важно заранее определить, какие данные должны сохраняться и как они будут использоваться при обработке событий.

Отдельного внимания требует организация логики переходов между этапами сценария. Одно и то же сообщение пользователя может иметь разный смысл в зависимости от текущего состояния. Например, текстовое сообщение может быть ответом на вопрос теста, описанием заявки или выбором

пункта меню. Поэтому для построения сценарных ботов часто используется механизм состояний, позволяющий определить, на каком этапе взаимодействия находится пользователь.

Разработка Telegram-бота без специальных инструментов требует от пользователя знания языка программирования, понимания принципов работы Telegram Bot API, настройки обработчиков сообщений и организации хранения данных. В официальных материалах Telegram также указывается, что после создания бота через BotFather необходимо подключить его к серверной части приложения, которая будет обрабатывать запросы и выполнять действия [16].

Таким образом, разработка Telegram-ботов включает несколько основных этапов: регистрацию бота через BotFather, получение токена, создание серверной части приложения, настройку получения обновлений, обработку сообщений и кнопок, хранение состояния пользователя и отправку ответов. Эти особенности обосновывают необходимость создания инструментария, который позволяет скрыть техническую сложность разработки и предоставить пользователю готовые элементы для построения сценария Telegram-бота.

1.2 Обзор существующих решений для no-code создания Telegram-ботов

На сегодняшний день в сети Интернет существует достаточно большое количество сервисов, предоставляющих возможность создавать чат-ботов и Telegram-ботов без ручного написания программного кода. Такие сервисы ориентированы на пользователей, которым требуется автоматизировать общение с клиентами, организовать справочный сервис, создать меню, настроить рассылки или реализовать простой сценарий взаимодействия с пользователем.

Под no-code подходом понимается способ разработки, при котором пользователь может создавать приложения и автоматизировать процессы без написания программного кода, используя визуальные интерфейсы и готовые элементы [5]. В рамках данной работы no-code конструктор ботов рассматривается как программная система, позволяющая создавать логику чат-бота с помощью сообщений, кнопок, блоков, условий, переходов и интеграций. Основная функция таких систем заключается в снижении сложности разработки и предоставлении пользователю возможности собрать бота без знания языка программирования. В данном разделе представлен обзор наиболее известных решений, с помощью которых пользователь может создавать Telegram-ботов или чат-ботов с поддержкой Telegram. При анализе учитывались следующие критерии:

1. возможность создания бота без написания программного кода;
2. наличие визуального или сценарного редактора;
3. поддержка кнопок, меню и переходов между этапами сценария;
4. наличие дополнительных функций: интеграций, рассылок, платежей или искусственного интеллекта;
5. ограничения бесплатного использования.

В результате для подробного анализа были отобраны следующие системы:

- Botmother;
- Aimylogic;
- SendPulse;
- Manybot.

Ниже представлено краткое описание приведённых систем и их основных функциональных возможностей.

Botmother

Botmother представляет собой no-code платформу для создания чат-ботов. Сервис поддерживает работу с несколькими каналами взаимодействия, включая Telegram, WhatsApp, VK и другие платформы. Согласно описанию официального сайта, логика бота собирается в визуальном редакторе из готовых блоков без написания программного кода [1].

Основная функция Botmother заключается в создании чат-ботов для автоматизации общения с пользователями. Сервис позволяет собирать сценарии взаимодействия, настраивать сообщения, использовать кнопки, подключать внешние сервисы и выстраивать последовательность действий пользователя.

Пользователь имеет возможность:

- создавать бота без написания программного кода;
- использовать визуальный редактор сценариев;
- подключать различные каналы взаимодействия;
- настраивать сообщения, кнопки и переходы между блоками;
- использовать интеграции с внешними сервисами;
- применять дополнительные компоненты, включая платежи.

Необходимо отметить, что Botmother ориентирован преимущественно на создание бизнес-ботов. Сервис может применяться для консультаций, автоматизации продаж, обработки обращений и информирования клиентов. К преимуществам решения относится широкий набор готовых возможностей и поддержка нескольких мессенджеров.

Вместе с тем Botmother является внешней платформой, поэтому использование созданного бота зависит от условий сервиса и доступного набора функций. Данное решение удобно для быстрого создания чат-ботов,

однако не предполагает полного контроля пользователя над внутренней архитектурой системы.

Aimylogic

Aimylogic является платформой для создания чат-ботов и голосовых роботов. Данное решение ориентировано на автоматизацию бизнес-процессов, поддержку клиентов, обработку типовых обращений и создание диалоговых систем.

Основной особенностью Aimylogic является ориентация на более сложные сценарии общения, в том числе с использованием искусственного интеллекта. По информации официальной документации, бесплатный план сервиса имеет ограничения по числу уникальных пользователей, а ChatGPT и дополнительные пакеты недоступны на бесплатном плане [2].

Пользователь имеет возможность:

- создавать чат-ботов и голосовых роботов;
- настраивать диалоговые сценарии;
- подключать различные каналы взаимодействия;
- использовать интеграции и API;
- применять функции искусственного интеллекта в рамках доступного тарифа.

Преимуществом Aimylogic является возможность разработки более гибких диалоговых сценариев. Это позволяет применять сервис для консультантов, клиентской поддержки и автоматизации обращений.

Однако необходимо отметить, что Aimylogic может быть избыточным для создания простых Telegram-ботов с меню, кнопками и переходами. Кроме того, использование сервиса связано с тарифными ограничениями. Та-

ким образом, платформа подходит для бизнес-задач и интеллектуальных диалогов, но не всегда является оптимальным решением для простых сценарных Telegram-ботов.

SendPulse

SendPulse представляет собой платформу для маркетинговой автоматизации, в состав которой входит визуальный конструктор чат-ботов. Сервис поддерживает создание Telegram-чат-ботов и позволяет настраивать автоматические ответы, цепочки сообщений, рассылки и платежные сценарии [3].

Основная функция SendPulse заключается в автоматизации коммуникаций с подписчиками и клиентами. Сервис позволяет создавать цепочки сообщений, проводить рассылки, управлять подписчиками и связывать чат-бота с другими инструментами платформы.

Пользователь имеет возможность:

- создавать Telegram-бота с помощью визуального конструктора;
- использовать цепочки сообщений;
- настраивать автоматические ответы и рассылки;
- работать с подписчиками;
- подключать дополнительные инструменты платформы.

Преимуществом SendPulse является развитая экосистема маркетинговых инструментов. Сервис может быть полезен для компаний, которым требуется не только бот, но и рассылки, работа с клиентской базой и автоматизация продаж.

Ограничением SendPulse является ориентация на маркетинговые задачи. Для пользователя, которому необходимо создать небольшой сценарный Telegram-бот, часть функций может быть избыточной. Кроме того, расширенное использование сервиса зависит от условий выбранного тарифа.

Manybot

Manybot является сервисом, ориентированным на создание Telegram-ботов без программирования. В отличие от Botmother, Aimylogic и SendPulse, данный инструмент реализован в формате Telegram-бота, через который пользователь создаёт и настраивает собственного бота.

Согласно описанию официального сайта, Manybot позволяет создавать Telegram-ботов без программирования, отправлять сообщения подписчикам, настраивать пользовательские команды, а также создавать многоуровневые меню и подменю [4].

Пользователь имеет возможность:

- создать Telegram-бота без программирования;
- настроить пользовательские команды;
- создать меню и подменю;
- отправлять сообщения подписчикам;
- добавить администраторов для управления ботом.

Manybot подходит для создания простых информационных Telegram-ботов, меню-ботов и ботов для рассылки сообщений подписчикам. Его преимуществом является простота использования и ориентация именно на Telegram.

Вместе с тем возможности Manybot ограничены заранее предусмотренным набором функций. Сервис удобен для простых меню и команд, но менее подходит для построения сложной сценарной логики. Поэтому Manybot можно рассматривать как удобный инструмент для простых Telegram-ботов, но не как универсальное решение для создания ботов с расширенной логикой поведения.

1.3 Обзор инструментальных средств и технологий реализации no-code инструментария для Telegram-ботов

1.3.1 Анализ языковых средств разработки

На сегодняшний день существует достаточно большой выбор языковых средств, которые могут применяться для разработки Telegram-ботов и программных инструментов для их создания. Выбор языка программирования влияет на архитектуру приложения, удобство обработки пользовательских сообщений, возможность работы с базой данных, поддержку асинхронного выполнения и сложность дальнейшего сопровождения проекта.

К наиболее подходящим языковым средствам для разработки Telegram-ботов можно отнести:

- Python;
- JavaScript / TypeScript;
- Java;
- C#.

Для реализации дипломного проекта был выбран язык программирования Python. Данный выбор обусловлен тем, что Python обладает низким порогом входа, большим количеством библиотек для работы с Telegram Bot API, поддерживает асинхронное программирование и широко применяется для разработки серверных приложений, автоматизации и прикладных программных систем. Официальная документация Python содержит описание синтаксиса языка, стандартной библиотеки и основных механизмов разработки приложений [6].

Python является интерпретируемым языком программирования высокого уровня. Одним из его преимуществ является читаемый синтаксис, позволяющий быстрее разрабатывать и изменять программный код. Это особенно важно при создании учебного проекта, в котором требуется не только

реализовать работающее приложение, но и обеспечить понятность архитектуры и возможность дальнейшего расширения.

Важной особенностью Python является наличие развитой экосистемы библиотек. Для разработки Telegram-ботов доступны различные библиотеки, позволяющие обрабатывать сообщения, команды, callback-запросы, клавиатуры и состояния пользователя. Кроме того, Python предоставляет удобные средства для работы с базами данных, файлами, журналированием событий и асинхронным выполнением операций.

Отдельным преимуществом Python при разработке Telegram-ботов является поддержка асинхронного программирования. Модуль `asyncio` предназначен для написания конкурентного кода с использованием синтаксиса `async/await` и применяется при создании сетевых и событийных приложений [17]. Это важно для Telegram-ботов, так как программа должна обрабатывать сообщения, команды и нажатия кнопок от разных пользователей без блокировки основного процесса работы.

JavaScript и TypeScript также могут применяться для разработки Telegram-ботов. Их преимуществом является широкое распространение в веб-разработке и возможность использования единого языка как на клиентской, так и на серверной стороне. Однако в рамках данной работы разработка отдельного веб-интерфейса не является основной задачей, поэтому применение JavaScript или TypeScript не являлось обязательным.

Java и C# являются распространёнными языками для разработки серверных и корпоративных приложений. Они обладают развитой экосистемой, строгой типизацией и большим количеством инструментов для промышленной разработки. Вместе с тем для реализации сравнительно небольшого инструментария создания Telegram-ботов их использование может привести к увеличению объёма кода и усложнению архитектуры проекта.

Таким образом, для реализации дипломного проекта был выбран язык Python, так как он сочетает простоту разработки, наличие библиотек для Telegram-ботов, поддержку асинхронного программирования и удобство сопровождения программного кода.

1.3.2 Анализ фреймворков, библиотек и среды разработки

В процессе анализа инструментальных средств для реализации проекта рассматривались библиотеки и средства, необходимые для решения следующих задач:

- взаимодействие с Telegram Bot API;
- обработка сообщений, команд и нажатий кнопок;
- организация асинхронного выполнения операций;
- работа с файлами;
- журналирование событий;
- работа с базой данных через объектную модель;
- разработка и отладка программного кода.

Для разработки Telegram-ботов на Python существуют различные библиотеки. В официальных материалах Telegram среди примеров библиотек для Python указаны, в частности, `aiogram`, `python-telegram-bot` и `pyTelegramBotAPI` [7]. Данные библиотеки предоставляют разработчику средства для взаимодействия с Telegram Bot API, однако различаются архитектурой, подходом к обработке событий и удобством использования в асинхронных приложениях.

В рамках дипломного проекта в качестве основной библиотеки для разработки Telegram-бота была выбрана библиотека `aiogram`.

aiogram

`aiogram` — это асинхронный фреймворк для разработки Telegram-ботов на языке Python. Согласно официальной документации, `aiogram` является современным полностью асинхронным фреймворком для Telegram Bot API, использующим `asyncio` и `aiohttp` [8].

Основными преимуществами `aiogram` являются:

- асинхронная обработка входящих событий;
- поддержка команд, сообщений и `callback`-запросов;
- возможность работы с клавиатурами;
- поддержка маршрутизации обработчиков;
- наличие механизмов для построения диалоговой логики;
- удобная организация структуры проекта.

В `aiogram` обработка входящих событий строится на использовании диспетчера, маршрутизаторов, фильтров и обработчиков. `Dispatcher` и `Router` применяются для маршрутизации обновлений, поступающих от Telegram, а фильтры позволяют направлять сообщения и `callback`-запросы к нужным обработчикам [22], [23], [24]. Также `aiogram` предоставляет средства для работы с клавиатурами, что важно при создании сценарных ботов с кнопками и меню [25]. Для построения диалоговой логики может использоваться механизм конечных автоматов состояний, позволяющий хранить текущий этап взаимодействия пользователя с ботом [26].

Выбор `aiogram` обусловлен тем, что разрабатываемый инструментарий должен обрабатывать множество пользовательских событий: сообщения, команды, нажатия кнопок и переходы между этапами сценария. Асинхронная модель позволяет эффективнее организовать такую обработку и не блокировать работу приложения при выполнении отдельных операций.

Loguru

Для журналирования событий в проекте используется библиотека Loguru. Журналирование необходимо для отслеживания работы приложения, фиксации ошибок, анализа действий пользователя и упрощения процесса отладки.

Необходимость журналирования связана с тем, что в процессе работы программы могут возникать ошибки обработки сообщений, подключения к базе данных или выполнения действий сценария. В официальной документации Python журналирование рассматривается как средство фиксации событий, возникающих во время выполнения программы [18]. Поэтому использование отдельного инструмента логирования позволяет упростить отладку и сопровождение программного продукта.

Loguru представляет собой библиотеку для организации логирования в Python. В официальной документации указывается, что её задача заключается в упрощении настройки и использования журналирования по сравнению со стандартными средствами Python [9].

Основными преимуществами Loguru являются:

- простая настройка;
- удобный вывод сообщений журнала;
- возможность записи логов в файл;
- поддержка различных уровней сообщений;
- удобство применения при отладке приложения.

Использование Loguru позволяет упростить контроль работы системы и быстрее выявлять ошибки, возникающие при обработке сообщений, создании сценариев или взаимодействии с базой данных.

aiofiles

Для асинхронной работы с файлами используется библиотека `aiofiles`. В асинхронных приложениях обычные операции чтения и записи файлов могут блокировать выполнение программы. Это особенно важно для Telegram-бота, который должен продолжать обрабатывать входящие сообщения пользователей.

`aiofiles` предназначена для работы с локальными файлами в `asycio`-приложениях. Согласно описанию проекта, библиотека позволяет выполнять операции с файлами в асинхронном стиле [10].

К основным возможностям `aiofiles` относятся:

- асинхронное открытие файлов;
- асинхронное чтение и запись данных;
- использование совместно с `asycio`;
- снижение риска блокировки основного цикла событий приложения.

Применение `aiofiles` целесообразно в тех случаях, когда приложению необходимо работать с файлами конфигурации, временными файлами или другими данными, не блокируя обработку пользовательских сообщений.

Tortoise ORM

Для работы с данными в проекте используется Tortoise ORM. ORM позволяет работать с таблицами базы данных через классы и объекты Python, не формируя все SQL-запросы вручную. Это упрощает разработку и делает код более читаемым.

Tortoise ORM является асинхронной ORM-библиотекой для Python. В официальной документации указано, что она была разработана как лёгкий `async-native Object-Relational Mapper` с API, похожим на Django ORM [11].

Основными преимуществами Tortoise ORM являются:

- асинхронная работа с базой данных;

- описание моделей данных в виде Python-классов;
- поддержка связей между моделями;
- возможность выполнения запросов через объектный интерфейс;
- удобство интеграции с асинхронными Python-приложениями.

При разработке небольших прикладных систем и учебных проектов для хранения данных может использоваться SQLite. В документации Python указывается, что SQLite представляет собой легковесную файловую базу данных, не требующую отдельного серверного процесса [19]. Официальный сайт SQLite также характеризует SQLite как самодостаточный и встраиваемый SQL-движок [20]. Такой вариант подходит для хранения структуры создаваемых ботов, состояний, узлов, сообщений, клавиатур, триггеров и действий.

Tortoise ORM позволяет описывать модели данных в виде Python-классов, а также использовать связи между сущностями [29]. Это соответствует структуре разрабатываемой системы, так как созданный бот связан с состояниями, узлами, триггерами и действиями. Использование ORM упрощает работу с такими связями и снижает необходимость ручного написания SQL-запросов.

Официальные примеры Tortoise ORM демонстрируют использование моделей, подключение к базе данных и выполнение запросов через объектный интерфейс [30].

В рамках разрабатываемого инструментария Tortoise ORM может использоваться для хранения информации о пользователях, созданных ботах, сценариях, состояниях, действиях и других сущностях системы. Использование ORM позволяет отделить логику работы приложения от низкоуровневых SQL-запросов.

PyCharm

В качестве среды разработки используется PyCharm. PyCharm представляет собой интегрированную среду разработки для языка Python. На официальном сайте JetBrains указывается, что PyCharm поддерживает Python, базы данных, системы контроля версий и различные инструменты разработки [12].

К основным возможностям PyCharm относятся:

- подсветка синтаксиса Python;
- автодополнение кода;
- встроенные средства отладки;
- работа с виртуальными окружениями;
- интеграция с системами контроля версий;
- поддержка работы с базами данных;
- инструменты навигации по проекту.

При разработке проекта также может использоваться виртуальное окружение Python, позволяющее изолировать зависимости конкретного приложения от системной установки языка [21].

Использование PyCharm упрощает разработку проекта, так как позволяет работать с несколькими модулями, отслеживать ошибки, выполнять отладку и управлять структурой приложения в единой среде.

Выбранные средства соответствуют требованиям проекта, так как позволяют реализовать асинхронную обработку сообщений, работу с пользовательскими сценариями, хранение данных и журналирование событий.

Выводы к главе 1

1. Проанализированы особенности разработки Telegram-ботов. Установлено, что создание Telegram-бота связано с регистрацией бота через BotFather, получением токена, работой с Telegram Bot API, обработкой входящих событий, использованием сообщений, кнопок, клавиатур и хранением состояния пользователя.

2. Проведён анализ существующих решений в области no-code создания Telegram-ботов. В результате анализа были рассмотрены сервисы Botmother, Aimylogic, SendPulse и Manybot, выявлены их основные функциональные возможности, преимущества и ограничения.

3. Установлено, что существующие решения позволяют создавать Telegram-ботов без написания программного кода, однако большинство из них являются внешними платформами, использование которых зависит от тарифов, условий конкретного сервиса и заранее предусмотренного набора функций. Также часть рассмотренных систем ориентирована на более широкие задачи: маркетинговые рассылки, бизнес-коммуникации или интеллектуальные диалоги.

4. Проведён анализ языковых и инструментальных средств, которые могут быть использованы для реализации программного продукта. В качестве основного языка разработки выбран Python, что обусловлено его простотой, развитой экосистемой библиотек, поддержкой асинхронного программирования и удобством применения при разработке Telegram-ботов.

5. В качестве основной библиотеки для взаимодействия с Telegram Bot API выбрана aiogram. Для журналирования используется Loguru, для асинхронной работы с файлами — aiofiles, для организации работы с данными — Tortoise ORM. В качестве среды разработки используется PyCharm.

ГЛАВА 2. Проектирование и разработка программного продукта

2.1 Описание структуры и принципа работы программы

Разработанный программный продукт представляет собой инструментарий для no-code создания Telegram-ботов. Система позволяет пользователю создавать типовые сценарные Telegram-боты без ручного написания программного кода. Работа с программой осуществляется через управляющего Telegram-бота, который предоставляет пользователю интерфейс для создания и настройки будущих ботов.

Основная идея разработанного инструментария заключается в том, что логика Telegram-бота описывается не программным кодом, а набором готовых элементов. К таким элементам относятся бот, состояния, узлы, сообщения, клавиатуры, триггеры и действия. Пользователь создаёт эти элементы через интерфейс управляющего бота, а программа сохраняет структуру сценария и использует её при работе созданного Telegram-бота.

Разрабатываемая система состоит из двух основных частей: управляющего Telegram-бота и созданных пользовательских Telegram-ботов.

Управляющий Telegram-бот используется для настройки сценариев. Через него пользователь создаёт нового бота, указывает его данные, создаёт состояния, узлы, сообщения, клавиатуры, триггеры и действия. Все сведения о созданном боте и его сценарии сохраняются в базе данных.

Созданный пользовательский Telegram-бот выполняет уже настроенную логику. Когда конечный пользователь отправляет сообщение или нажимает кнопку, система определяет текущее состояние пользователя, находит подходящий триггер и выполняет связанное с ним действие.

В рамках разработанного программного продукта можно выделить несколько логических модулей, каждый из которых отвечает за отдельную часть работы системы:

- модуль управляющего Telegram-бота;
- модуль управления создаваемыми ботами;
- модуль настройки сценариев;
- модуль обработки пользовательских событий;
- модуль действий;
- модуль хранения данных;
- модуль клавиатур и пользовательского интерфейса;
- модуль конфигурации и журналирования.

Модуль управляющего Telegram-бота является основной частью системы, через которую пользователь взаимодействует с программой. Данный модуль предоставляет команды и кнопки для создания нового Telegram-бота, выбора существующего бота, настройки его структуры и редактирования отдельных элементов сценария. Управляющий бот выполняет роль интерфейса администратора, однако работа с ним осуществляется не через веб-страницу, а непосредственно в Telegram.

Модуль управления создаваемыми ботами отвечает за хранение и обработку сведений о ботах, которые создаются пользователем. В рамках данного модуля сохраняются основные данные создаваемого Telegram-бота, включая его название, токен и связь с пользователем-владельцем. Эти данные необходимы для дальнейшего запуска созданного бота и выполнения настроенного сценария.

Модуль настройки сценариев используется для формирования логики работы создаваемого бота. В этом модуле пользователь создаёт состояния и узлы сценария, связывает их с сообщениями, клавиатурами, триггерами и действиями. Благодаря этому поведение Telegram-бота задаётся не через программный код, а через набор связанных элементов, которые сохраняются в базе данных.

Модуль обработки пользовательских событий отвечает за реакцию созданного Telegram-бота на действия конечного пользователя. К таким действиям относятся отправка текстового сообщения и нажатие кнопки. После получения события система определяет, с каким ботом взаимодействует пользователь, в каком состоянии он находится и какой триггер должен быть применён. После этого выполняется действие, связанное с найденным триггером.

Модуль действий предназначен для выполнения операций, которые происходят после срабатывания триггера. В рамках разработанного инструментария действие может использоваться для отправки сообщения пользователю, изменения текущего состояния или передачи информации владельцу бота. Выделение действий в отдельную часть системы позволяет в дальнейшем расширять функциональность программы и добавлять новые типы операций.

Модуль хранения данных обеспечивает сохранение информации о пользователях, созданных ботах и элементах сценария. В базе данных хранятся сведения о ботах, состояниях, узлах, сообщениях, клавиатурах, триггерах и действиях. За счёт этого созданный пользователем сценарий не теряется после завершения работы программы и может быть использован при последующем запуске бота.

Необходимость хранения данных связана с тем, что создаваемый Telegram-бот должен сохранять свою структуру независимо от текущего сеанса работы пользователя с управляющим ботом. Пользователь может создать бота, настроить часть сценария, завершить работу, а затем вернуться к редактированию позднее. Для этого система должна хранить не только общие сведения о созданном боте, но и все элементы его сценария.

В системе можно выделить несколько групп сохраняемых данных. К первой группе относятся сведения о пользователе системы, который создаёт

и настраивает Telegram-ботов. Для такого пользователя сохраняется идентификатор Telegram и связь с созданными им ботами. Это позволяет системе определить, какие боты принадлежат конкретному пользователю, и предоставить ему доступ к их настройке.

Ко второй группе относятся сведения о создаваемых Telegram-ботах. Для каждого бота сохраняются основные данные, необходимые для его дальнейшей работы. К таким данным относятся название бота, токен, владелец и связь с элементами сценария. Токен используется для подключения созданного бота к Telegram Bot API, а остальные данные применяются для управления структурой сценария.

К третьей группе относятся элементы сценария. В рамках разработанного инструментария сценарий строится на основе состояний, узлов, сообщений, клавиатур, триггеров и действий. Каждый из этих элементов выполняет отдельную функцию. Состояния определяют этапы взаимодействия, узлы связывают элементы сценария, сообщения содержат текст ответа, клавиатуры задают кнопки, триггеры определяют условия срабатывания, а действия описывают реакцию бота.

Отдельное хранение элементов сценария позволяет изменять структуру бота без изменения исходного кода программы. Например, пользователь может изменить текст сообщения, добавить новую кнопку, создать дополнительное состояние или настроить другой переход между этапами сценария. При этом общая логика обработки остаётся единой для всех созданных ботов, а конкретное поведение каждого бота определяется сохранёнными данными.

Также в системе необходимо учитывать конечных пользователей созданного Telegram-бота. Для них может сохраняться текущее состояние, в котором они находятся. Это требуется для правильной обработки следующих сообщений или нажатий кнопок. Например, если пользователь находится на

этапе ввода заявки, отправленное им сообщение должно восприниматься как текст заявки, а не как произвольная команда.

Модуль клавиатур и пользовательского интерфейса отвечает за формирование кнопок, которые отображаются пользователю в Telegram. Клавиатуры применяются как в управляющем боте, так и в созданных пользовательских ботах. В управляющем боте они используются для навигации по разделам настройки, а в созданных ботах — для выбора вариантов ответа или перехода между этапами сценария.

Модуль конфигурации используется для хранения служебных параметров программы. К таким параметрам относятся настройки подключения, токены, параметры базы данных и другие данные, необходимые для запуска системы. Разделение конфигурации и основного программного кода упрощает сопровождение проекта.

Модуль журналирования предназначен для фиксации событий, возникающих во время работы программы. С его помощью можно отслеживать запуск системы, обработку сообщений, ошибки при выполнении действий и другие служебные события. Наличие журналирования упрощает поиск ошибок и отладку программы.

После описания основных модулей необходимо рассмотреть элементы, из которых строится сценарий создаваемого Telegram-бота. Основными элементами программы являются:

- бот — сущность, содержащая сведения о создаваемом Telegram-боте;
- состояние — этап взаимодействия пользователя с ботом;
- узел — элемент сценария, связывающий сообщение, клавиатуру, триггеры и действия;
- сообщение — текст, который бот отправляет пользователю;
- клавиатура — набор кнопок, отображаемых пользователю;

- триггер — условие, при котором бот должен выполнить определённую реакцию;
- действие — операция, выполняемая после срабатывания триггера.

Состояния используются для разделения сценария на этапы. Например, отдельными состояниями могут быть главное меню, этап ввода заявки, вопрос теста или завершающее сообщение. Благодаря состояниям бот может понимать, на каком этапе сценария находится пользователь.

Узел является основным элементом сценария. Он может содержать сообщение, клавиатуру и набор триггеров. Через узлы пользователь формирует последовательность работы создаваемого Telegram-бота.

Сообщение определяет текст, который бот должен отправить пользователю. При необходимости к сообщению прикрепляется клавиатура. Клавиатура позволяет пользователю выбирать доступные варианты действий с помощью кнопок.

Триггер определяет событие, на которое должен реагировать бот. В качестве такого события может выступать отправка текстового сообщения пользователем или нажатие кнопки. После срабатывания триггера выполняется связанное с ним действие.

Действие определяет реакцию бота. В рамках разработанного инструментария действие может использоваться для отправки сообщения пользователю, изменения состояния или передачи информации владельцу бота.

Взаимодействие между модулями и элементами сценария происходит следующим образом. Пользователь через управляющего Telegram-бота создаёт структуру будущего бота. Сведения о созданных элементах передаются в модуль хранения данных и сохраняются в базе. Когда конечный пользователь взаимодействует с созданным ботом, модуль обработки событий получает сообщение или нажатие кнопки, определяет текущее состояние пользо-

вателя, находит подходящий триггер и передаёт управление модулю действий. После выполнения действия пользователю отправляется ответ или изменяется его текущее состояние.

Принцип работы созданного Telegram-бота можно описать следующим образом. Сначала бот получает событие от пользователя: сообщение или нажатие кнопки. Затем система определяет текущее состояние пользователя и ищет триггер, соответствующий полученному событию. Если подходящий триггер найден, выполняется связанное с ним действие. В результате бот может отправить сообщение, изменить состояние пользователя или выполнить другую предусмотренную операцию.

Например, если пользователь нажимает кнопку «Оставить заявку», срабатывает триггер, связанный с этой кнопкой. После этого бот может отправить сообщение с просьбой ввести текст заявки и перевести пользователя в состояние ожидания ввода. Когда пользователь отправляет текстовое сообщение, система обрабатывает его как следующий этап сценария.

Таким образом, программа работает как исполнитель заранее настроенных сценариев. Пользователь системы создаёт структуру будущего Telegram-бота через управляющего бота, а созданный бот использует сохранённую структуру для обработки сообщений, кнопок и переходов между состояниями.

2.2 Функциональная модель и работа пользователя с системой

Функциональная модель системы описывает основные действия, которые пользователь может выполнять при работе с программным продуктом. В рамках данной системы основным пользователем является пользователь, создающий и настраивающий Telegram-бота через управляющего бота.

Пользователь системы взаимодействует с интерфейсом управляющего Telegram-бота и выполняет действия, связанные с созданием и настройкой сценария будущего бота. К таким действиям относятся создание Telegram-

бота, указание токена, создание состояний и узлов, настройка сообщений, клавиатур, триггеров и действий.

Для представления функциональной модели системы используется диаграмма вариантов использования. Диаграмма вариантов использования показывает, какие функции предоставляет система и какой внешний участник взаимодействует с этими функциями [27]. В рамках данной работы такая диаграмма применяется для отображения действий пользователя при создании и настройке Telegram-бота. UML является стандартным языком моделирования, предназначенным для визуализации, описания и документирования программных систем [28].



Рисунок 1 — Диаграмма вариантов использования системы

На диаграмме вариантов использования показаны основные функции, доступные пользователю системы. Пользователь может создать Telegram-бота, указать его токен, создать состояние, создать узел сценария, настроить

сообщение и клавиатуру, а также создать триггер и действие. Данные функции образуют основу процесса настройки сценарного Telegram-бота.

Основным вариантом использования системы является создание Telegram-бота. Пользователь запускает управляющего бота и выбирает действие, связанное с созданием нового бота. После этого система предлагает указать основные сведения о будущем боте. Одним из ключевых параметров является токен, полученный через BotFather. Без указания токена созданный бот не сможет взаимодействовать с Telegram.

Следующим вариантом использования является создание состояния. Состояние применяется для разделения сценария на отдельные этапы. Например, в FAQ-боте состоянием может быть главное меню, в заявочном боте — этап ввода обращения, а в боте для проведения тестирования — отдельный вопрос. Создание состояний позволяет программе определять, как необходимо обрабатывать действия пользователя на разных этапах сценария.

Создание узла сценария является следующим важным вариантом использования. Узел объединяет отдельные элементы логики бота. С его помощью пользователь связывает сообщение, клавиатуру, триггеры и действия. Благодаря этому сценарий можно представить как последовательность отдельных узлов, между которыми пользователь переходит при взаимодействии с ботом.

Настройка сообщения позволяет определить текст, который будет отправлен пользователю. Сообщение может использоваться для приветствия, вывода справочной информации, запроса данных, отображения вопроса теста или подтверждения выполненного действия. Текст сообщения является одним из основных элементов взаимодействия пользователя с ботом.

Настройка клавиатуры используется для добавления кнопок. Клавиатура упрощает взаимодействие с ботом, так как пользователь выбирает один из предложенных вариантов, а не вводит команду вручную. Это особенно

важно для сценарных ботов, где необходимо направлять пользователя по заранее заданному пути.

Создание триггера позволяет определить условие, при котором бот должен выполнить действие. Триггер может быть связан с нажатием кнопки или отправкой сообщения. Например, в FAQ-боте нажатие кнопки с названием раздела вызывает отправку справочного сообщения. В заявочном боте отправка текста может означать ввод обращения. В боте для проведения тестирования нажатие кнопки может означать выбор варианта ответа.

Создание действия завершает настройку реакции бота. Действие определяет, что произойдёт после срабатывания триггера. В зависимости от сценария действие может отправить сообщение, изменить состояние пользователя или передать информацию владельцу бота.

Работа пользователя с системой начинается с запуска управляющего Telegram-бота. После запуска пользователь получает доступ к основным разделам управления и может перейти к созданию нового бота или настройке уже созданного.

На первом этапе пользователь создаёт Telegram-бота и указывает его основные данные. Одним из обязательных параметров является токен, полученный через BotFather. Токен необходим для подключения создаваемого бота к Telegram Bot API и дальнейшей работы с ним.

На следующем этапе пользователь формирует структуру сценария. Для этого создаются состояния, которые соответствуют отдельным этапам взаимодействия с конечным пользователем. Например, отдельным состоянием может быть главное меню, этап ввода заявки, вопрос теста или завершающее сообщение.

После создания состояния пользователь создаёт узлы сценария. Узел используется для описания конкретного элемента логики бота. Он может быть связан с сообщением, клавиатурой, триггерами и действиями. Через

узлы пользователь формирует последовательность работы создаваемого Telegram-бота.

Далее выполняется настройка сообщения. Сообщение определяет текст, который бот должен отправить пользователю на определённом этапе сценария. При необходимости к сообщению может быть прикреплена клавиатура, содержащая кнопки для выбора дальнейшего действия.

После настройки сообщений и клавиатур пользователь создаёт триггеры. Триггер определяет условие, при котором бот должен выполнить определённую реакцию. В качестве такого условия может выступать отправка сообщения пользователем или нажатие кнопки.

Завершающим этапом является создание действия. Действие определяет, что должно произойти после срабатывания триггера. Например, бот может отправить сообщение, изменить состояние пользователя или выполнить другую операцию, предусмотренную системой.

После настройки основных элементов пользователь может проверить работу созданного Telegram-бота. Для этого он запускает созданного бота и проходит сценарий так, как это будет делать конечный пользователь. При необходимости пользователь возвращается к управляющему боту и изменяет отдельные элементы сценария.

Таким образом, функциональная модель системы отражает последовательность действий пользователя при создании сценарного Telegram-бота. Пользователь постепенно формирует структуру будущего бота, начиная с создания самого бота и заканчивая настройкой триггеров и действий. В результате формируется логика работы Telegram-бота, которая сохраняется в системе и может быть использована для обработки сообщений и нажатий кнопок без изменения исходного кода программы.

2.3 Проверка работы инструментария на типовых сценариях

Для проверки работоспособности разработанного инструментария были созданы типовые сценарии Telegram-ботов. В качестве таких сценариев выбраны FAQ-бот, простой заявочный бот и бот для проведения тестирования. Данные варианты были выбраны потому, что они отражают распространённые задачи, решаемые с помощью сценарных Telegram-ботов: предоставление справочной информации, приём обращений от пользователей и организация тестирования.

Каждый из рассмотренных ботов был создан с использованием общей модели системы. Логика их работы задавалась через состояния, узлы, сообщения, клавиатуры, триггеры и действия. Пользователь не изменял исходный код программы, а формировал сценарий через предусмотренные элементы инструментария.

FAQ-бот предназначен для предоставления пользователю справочной информации по заранее заданным разделам. Такой тип бота может применяться в организациях, учебных подразделениях, сервисных службах или сообществах, где пользователи часто задают однотипные вопросы.

Основная задача FAQ-бота заключается в том, чтобы заменить ручную выдачу типовой информации автоматизированным меню. Пользователь запускает бота, получает список доступных разделов и выбирает интересующий его пункт. После выбора бот отправляет заранее подготовленный ответ.

В рамках созданного сценария FAQ-бот содержит главное состояние, в котором пользователю отображается приветственное сообщение и клавиатура с доступными разделами. Каждый пункт клавиатуры связан с отдельным триггером. При нажатии на кнопку срабатывает соответствующий триггер, после чего выполняется действие отправки сообщения пользователю.

Рисунок 2 — Пример работы FAQ-бота

Простой заявочный бот предназначен для сбора обращения от пользователя и передачи результата владельцу или администратору бота. Такой сценарий может использоваться для приёма заявок на услугу, обращений в техническую поддержку, записи на консультацию или сбора обратной связи.

Основная задача заявочного бота заключается в том, чтобы провести пользователя по нескольким этапам и получить от него необходимую информацию. В простейшем варианте бот запрашивает тему обращения, контактные данные и описание проблемы. После завершения сценария бот отправляет пользователю подтверждающее сообщение.

В рамках созданного сценария заявочный бот использует несколько состояний. Каждое состояние соответствует отдельному этапу заполнения заявки. Например, на первом этапе пользователь выбирает тип обращения, на втором вводит имя или контакт, на третьем описывает проблему.

После нажатия кнопки «Оставить заявку» пользователь переводится в состояние ввода обращения. Отправленное пользователем сообщение обрабатывается как триггер, после чего выполняется действие, предусмотренное сценарием. В зависимости от настроек бот может отправить подтверждение пользователю и передать текст обращения владельцу бота.

Рисунок 3 — Пример работы простого заявочного бота

Бот для проведения тестирования предназначен для создания простых тестов, которые могут применяться, например, преподавателями для проверки знаний обучающихся. Такой бот позволяет задавать пользователю вопросы и предлагать варианты ответов с помощью кнопок.

Основная задача тестового бота заключается в последовательном прохождении пользователем набора вопросов. Каждый вопрос может быть представлен отдельным состоянием. Варианты ответов отображаются в виде клавиатуры. При выборе варианта ответа срабатывает триггер, после чего бот переводит пользователя к следующему вопросу или завершает тестирование.

В рамках созданного сценария бот для тестирования содержит несколько состояний, соответствующих вопросам теста. Для каждого состояния задаётся текст вопроса и клавиатура с вариантами ответа. После выбора ответа пользователь переводится к следующему вопросу. После прохождения последнего вопроса бот отправляет сообщение о завершении тестирования.

Рисунок 4 — Пример работы бота для проведения тестирования

Создание данных сценариев позволило проверить основные функции разработанного инструментария: создание Telegram-бота, настройку состояний, узлов, сообщений, клавиатур, триггеров и действий, а также переходы между этапами сценария.

Выводы к главе 2

1. Описана структура разработанного инструментария для no-code создания Telegram-ботов. Установлено, что работа программы основана на использовании управляющего Telegram-бота, через который пользователь создаёт и настраивает сценарии будущих ботов.

2. Рассмотрен принцип работы программного продукта. Логика создаваемого Telegram-бота описывается с помощью состояний, узлов, сообщений, клавиатур, триггеров и действий, что позволяет формировать сценарий работы бота без изменения исходного кода программы.

3. Описана функциональная модель системы и основные этапы работы пользователя с программой. Пользователь может создать Telegram-бота, указать его токен, сформировать состояния и узлы, настроить сообщения, клавиатуры, триггеры и действия.

4. Проведена проверка работы разработанного инструментария на типовых сценариях. В качестве примеров были созданы FAQ-бот, простой заявочный бот и бот для проведения тестирования, что позволило проверить создание сценариев, обработку сообщений и кнопок, а также переходы между состояниями.

ЗАКЛЮЧЕНИЕ

При выполнении данной дипломной работы был разработан программный продукт — инструментарий для no-code создания Telegram-ботов на языке Python, который позволяет пользователю создавать типовые сценарные Telegram-боты без написания программного кода.

В процессе выполнения работы были решены следующие задачи:

- проанализированы особенности разработки Telegram-ботов и существующие решения в области создания ботов без программирования;
- проанализированы и на основе анализа отобраны адекватные задаче инструментальные средства;
- описана структура и принцип работы программного продукта, основанного на использовании состояний, узлов, сообщений, клавиатур, триггеров и действий;
- в соответствии с разработанной моделью реализован инструментарий и проведена проверка его работы на типовых сценариях создания Telegram-ботов.

Разработанный инструментарий позволяет создавать Telegram-ботов, настраивать состояния, узлы, сообщения, клавиатуры, триггеры и действия. Работа с системой осуществляется через управляющего Telegram-бота, что позволяет пользователю формировать сценарий будущего бота без изменения исходного кода программы.

Проверка работы системы была выполнена на сценариях создания FAQ-бота, простого заявочного бота и бота для проведения тестирования. В ходе проверки были рассмотрены создание сценариев, обработка сообщений и кнопок, а также переходы между состояниями.

Таким образом, поставленные задачи реализованы, а цель дипломной работы достигнута.

В будущем возможно дальнейшее совершенствование созданного продукта: расширение набора доступных действий, добавление готовых шаблонов ботов, улучшение интерфейса настройки сценариев, развитие механизмов хранения пользовательских данных, а также добавление интеграций с внешними сервисами.

ЛИТЕРАТУРА

1. Botmother. No-code Chatbot Builder For Business [Электронный ресурс]. – URL: <https://botmother.com/> (дата обращения: 14.05.2026). – Режим доступа: свободный.
2. Aimylogic subscription plans. Conversational Cloud Help Center [Электронный ресурс]. – URL: <https://help.cloud.just-ai.com/en/aimylogic/overview/pricing/> (дата обращения: 21.05.2026). – Режим доступа: свободный.
3. SendPulse. Create Telegram Chatbot for Free! [No-Code] [Электронный ресурс]. – URL: <https://sendpulse.com/features/chatbot/telegram> (дата обращения: 15.03.2026). – Режим доступа: свободный.
4. Manybot. Create a Telegram bot without coding [Электронный ресурс]. – URL: <https://manybot.io/> (дата обращения: 16.05.2026). – Режим доступа: свободный.
5. IBM. What is no-code? [Электронный ресурс]. – URL: <https://www.ibm.com/think/topics/no-code> (дата обращения: 10.04.2026). – Режим доступа: свободный.
6. Python Documentation. Официальная документация Python [Электронный ресурс]. – URL: <https://docs.python.org/3/> (дата обращения: 14.03.2026). – Режим доступа: свободный.
7. Telegram Bot API Library Examples. Официальные примеры библиотек для Telegram Bot API [Электронный ресурс]. – URL: <https://core.telegram.org/bots/samples> (дата обращения: 14.03.2026). – Режим доступа: свободный.
8. aiogram Documentation. Официальная документация aiogram [Электронный ресурс]. – URL: <https://docs.aiogram.dev/> (дата обращения: 16.04.2026). – Режим доступа: свободный.

9. Loguru Documentation. Официальная документация Loguru [Электронный ресурс]. – URL: <https://loguru.readthedocs.io/> (дата обращения: 18.04.2026). – Режим доступа: свободный.
10. aiofiles. Описание проекта aiofiles [Электронный ресурс]. – URL: <https://github.com/Tinche/aiofiles> (дата обращения: 17.04.2026). – Режим доступа: свободный.
11. Tortoise ORM Documentation. Официальная документация Tortoise ORM [Электронный ресурс]. – URL: <https://tortoise.github.io/> (дата обращения: 15.04.2026). – Режим доступа: свободный.
12. PyCharm. Официальная страница JetBrains PyCharm [Электронный ресурс]. – URL: <https://www.jetbrains.com/pycharm/> (дата обращения: 19.04.2026). – Режим доступа: свободный.
13. Telegram Bot API. Официальная документация Telegram [Электронный ресурс]. – URL: <https://core.telegram.org/bots/api> (дата обращения: 16.05.2026). – Режим доступа: свободный.
14. From BotFather to "Hello World". Официальное руководство Telegram [Электронный ресурс]. – URL: <https://core.telegram.org/bots/tutorial> (дата обращения: 19.02.2026). – Режим доступа: свободный.
15. Telegram Bot Features. Официальная документация Telegram [Электронный ресурс]. – URL: <https://core.telegram.org/bots/features> (дата обращения: 19.03.2026). – Режим доступа: свободный.
16. Bots FAQ. Официальная документация Telegram [Электронный ресурс]. – URL: <https://core.telegram.org/bots/faq> (дата обращения: 23.05.2026). – Режим доступа: свободный.
17. Python Documentation. asyncio — Asynchronous I/O [Электронный ресурс]. – URL: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 23.05.2026). – Режим доступа: свободный.
18. Python Documentation. Logging HOWTO [Электронный ресурс]. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 9.02.2026). – Режим доступа: свободный.

19. Python Documentation. sqlite3 — DB-API 2.0 interface for SQLite databases [Электронный ресурс]. – URL: <https://docs.python.org/3/library/sqlite3.html> (дата обращения: 17.04.2026). – Режим доступа: свободный.
20. SQLite. Official Website [Электронный ресурс]. – URL: <https://sqlite.org/> (дата обращения: 10.06.2026). – Режим доступа: свободный.
21. Python Documentation. venv — Creation of virtual environments [Электронный ресурс]. – URL: <https://docs.python.org/3/library/venv.html> (дата обращения: 27.04.2026). – Режим доступа: свободный.
22. aiogram Documentation. Handling events [Электронный ресурс]. – URL: <https://docs.aiogram.dev/en/latest/dispatcher/index.html> (дата обращения: 24.03.2026). – Режим доступа: свободный.
23. aiogram Documentation. Router [Электронный ресурс]. – URL: <https://docs.aiogram.dev/en/latest/dispatcher/router.html> (дата обращения: 22.05.2026). – Режим доступа: свободный.
24. aiogram Documentation. Filtering events [Электронный ресурс]. – URL: <https://docs.aiogram.dev/en/latest/dispatcher/filters/index.html> (дата обращения: 22.05.2026). – Режим доступа: свободный.
25. aiogram Documentation. Keyboard builder [Электронный ресурс]. – URL: <https://docs.aiogram.dev/en/v3.22.0/utils/keyboard.html> (дата обращения: 21.05.2026). – Режим доступа: свободный.
26. aiogram Documentation. Finite State Machine [Электронный ресурс]. – URL: https://docs.aiogram.dev/en/v3.20.0/dispatcher/finite_state_machine/ (дата обращения: 23.05.2026). – Режим доступа: свободный.
27. PlantUML. Use Case Diagram [Электронный ресурс]. – URL: <https://plantuml.com/use-case-diagram> (дата обращения: 16.03.2026). – Режим доступа: свободный.
28. Object Management Group. Unified Modeling Language [Электронный ресурс]. – URL: <https://www.omg.org/spec/UML/> (дата обращения: 18.04.2026). – Режим доступа: свободный.

29. Tortoise ORM Documentation. Models [Электронный ресурс]. – URL: <https://tortoise.github.io/models.html> (дата обращения: 3.02.2026). – Режим доступа: свободный.

30. Tortoise ORM Documentation. Examples [Электронный ресурс]. – URL: <https://tortoise.github.io/examples.html> (дата обращения: 25.03.2026). – Режим доступа: свободный.